

Report on

GAME OF THORNS PERSONALITY MATCHER

Submitted to:

University Institute of Computing

Submitted by:

Ayan Pal (25MCD10071)



Chandigarh University, Mohali

BONAFIDE CERTIFICATE

Certified that this project report "**Game Of Thrones Personality Matcher**" is the Bonafide work Of "**Ayan Pal**" who carried out the project work under my/our supervision.

SIGNATURE

Dr. Amandeep Kaur
HEAD OF THE DEPARTMENT
(University Institute of Computing)

SIGNATURE

Dr. Lovekesh Singh
(Assistant professor)
(University Institute of Computing)

Submitted for the project viva-voce examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

**AYAN PAL** 🇮🇳

@ayan73144



Complete your profile

Add your missing details →

This data will be helpful to auto-fill your job applications

**Personal Information**

📧 ayan73144@gmail.com

📞 Add your mobile number

📍 India

My Resume[Ayan_Pal_Resume_C2.pdf](#)

EEO settings ⓘ

**My Badges****My Certifications**You have not earned any certifications yet. [Get Certified](#)**Work Experience**[+ Add Work Experience](#)

Add your work experience. Don't forget to add those internships as well.

Education[+ Add Education](#)

We believe in skills over pedigree, but go ahead add your education for the recruiters who don't.

Abstract

This project, titled "Game of Thrones Personality Matcher," is an NLP-based content recommendation system that identifies which Game of Thrones character speaks in a manner most similar to a character selected by the user. The system works by grouping all dialogue lines from the show's script dataset by character, converting each character's combined dialogue into a numeric vector using the Bag of Words technique via scikit-learn's CountVectorizer, and then reducing these high-dimensional vectors to two dimensions using t-SNE (t-distributed Stochastic Neighbor Embedding) for visualization and comparison. Similarity between characters is measured using Euclidean distance on the reduced 2D coordinates, and the character with the smallest distance to the selected one is returned as the recommended match. The entire pipeline is deployed as an interactive Streamlit web application, where a user selects a character from a dropdown and instantly views the recommended match along with character portraits fetched from a public API. The project uses the "Game of Thrones Data" dataset from Kaggle, restricting analysis to the top 100 (primarily top 25) characters by dialogue volume to ensure statistically meaningful vectors. While the system does not account for word order, context, tone, or personality traits in the way a human viewer would define them, it demonstrates a complete, practical unsupervised NLP pipeline covering data cleaning, feature extraction, dimensionality reduction, similarity computation, and web deployment. The project is intended as a learning exercise in fundamental text vectorization and recommendation system concepts, with identified scope for future improvement using more advanced embedding techniques such as TF-IDF, Word2Vec, or large language model-based approaches.

Table of Contents

Table of Contents.....	1
Chapter 1 - Introduction	7
1.1 Introduction	7
1.2 Background	7
1.3 Need of the Project.....	7
1.4 Problem Statement.....	8
1.5 Objectives.....	8
1.6 Scope.....	8
1.7 Applications.....	8
1.8 Advantages.....	8
1.9 Limitations.....	9
Chapter 2 - Literature Survey.....	10
2.1 Natural Language Processing.....	10
2.2 Recommendation Systems.....	10
2.3 Bag of Words.....	10
2.4 CountVectorizer	11
2.5 t-SNE.....	11
2.6 Existing Recommendation Systems	11
2.7 Comparison Table	12
Chapter 3 - System Analysis.....	13
3.1 Existing System.....	13
3.2 Proposed System.....	13
3.3 Functional Requirements.....	13
3.4 Non-functional Requirements	13
3.5 Software Requirements	13
3.6 Hardware Requirements.....	14
3.7 Feasibility Study	14
Chapter 4 - Dataset Description.....	16
4.1 Kaggle Dataset	16
4.2 Why This Dataset Was Selected.....	16
4.3 Dialogue Dataset.....	16
4.4 Data Cleaning	16
4.5 Data Preprocessing	16
4.6 Feature Extraction.....	17
Chapter 5 - Methodology.....	18

5.1 Bag of Words.....	18
5.2 CountVectorizer	18
5.3 Vector Representation	18
5.4 t-SNE.....	19
5.5 Euclidean Distance	19
5.6 Similarity Recommendation Algorithm.....	19
Chapter 6 - Implementation	22
6.1 Pandas	22
6.2 NumPy.....	22
6.3 Requests.....	22
6.4 Pickle	22
6.5 Streamlit.....	22
6.6 Plotly	23
6.7 Scikit-learn	23
6.8 Important Code Snippet - Recommendation Logic.....	23
6.9 Character Name Fixing.....	23
Chapter 7 - Results and Discussion	26
7.1 Input.....	26
7.2 Output.....	26
7.3 Character Recommendation	26
7.4 Performance	26
7.5 Observations	26
Chapter 8 - Future Scope	28
8.1 Movie Recommendation.....	28
8.2 Music Recommendation	28
8.3 Book Recommendation.....	28
8.4 Customer Segmentation	28
8.5 Modern NLP Embeddings	28
8.6 Large Language Models	28
Chapter 9 - Conclusion	29
References	Error! Bookmark not defined.
Appendix	Error! Bookmark not defined.
A.1 Project Folder Structure	Error! Bookmark not defined.
A.2 requirements.txt.....	Error! Bookmark not defined.
A.3 Important Code Snippets.....	Error! Bookmark not defined.

Chapter 1 - Introduction

1.1 Introduction

This project, "Game of Thrones Personality Matcher," is a small NLP based recommendation system that I built to understand how text data can be converted into something a machine can compare mathematically. The basic idea is simple - you pick a Game of Thrones character from a dropdown, and the app tells you which other character talks the most like them, based on the actual dialogue lines from the show. It sounds like a fun little side project, and it is, but underneath it uses some real data science concepts like Bag of Words, CountVectorizer and t-SNE, which are actually used in serious recommendation systems as well.

I picked this topic because I wanted a project where I could see the results in a visual and interesting way, rather than just printing numbers on a screen. Working with GoT dialogues made the whole learning process a lot more enjoyable, and at the same time it forced me to deal with real messy text data, which is what most NLP projects are about anyway.

Another reason I liked this idea is that it is very easy to explain to someone who has never studied data science. If I say "the model does content-based filtering using vectorized text similarity", most people's eyes will glaze over. But if I say "it looks at how much a character talks and about what, and then finds another character who talks in a similar way", that makes immediate sense even to someone with zero technical background. This made the project a good balance between something technically valid and something that is genuinely fun to demonstrate.

1.2 Background

Natural Language Processing (NLP) is the branch of computer science (and specifically AI) that deals with how computers understand and process human language. It is used everywhere now - spam filters, chatbots, translation apps, voice assistants, and recommendation engines. Most people don't realize that behind Netflix suggesting a movie or Spotify suggesting a song, there is some form of similarity calculation happening, very similar in spirit to what this project does but obviously at a much bigger scale.

Recommendation systems generally work by representing items (movies, songs, products, or in this case, characters) as vectors of numbers, and then measuring how close or far these vectors are from each other. Characters whose vectors are close together are considered similar. This project uses that same core idea but applies it to something unusual - comparing how TV characters speak.

It is worth mentioning that "similarity" in this project has a very specific and narrow meaning. It does not mean the characters have similar personalities in the way a fan would describe them (brave, cunning, loyal, etc). It only means their choice of words, based on raw frequency, places them close together mathematically. This distinction is important to keep in mind throughout the report, because it explains both why some recommendations feel spot on and why some feel a little off.

1.3 Need of the Project

Most college-level NLP projects use very standard datasets like movie reviews or spam emails, and while those are useful, they get repetitive. I wanted to work with a dataset that is not the usual one, and Game of Thrones dialogue data fits well because:

- It is real conversational text, not just formal writing, so it has more variety in vocabulary and sentence style.
- Characters have very distinct speaking styles, so there is a genuine chance of finding meaningful similarity.

- It makes the whole exercise more engaging, both for building it and for explaining it to others.

1.4 Problem Statement

Given a dataset of dialogue lines spoken by different characters in Game of Thrones, the goal is to build a system that can take one character as input and return the character whose dialogue pattern is closest to it in terms of word usage. This needs to be done using unsupervised NLP techniques since there are no labelled "similar pairs" given anywhere - the system has to figure out similarity purely from the text itself.

1.5 Objectives

1. To collect and clean dialogue data of GoT characters from the given dataset.
2. To convert each character's dialogue into a numeric vector representation using Bag of Words.
3. To reduce the dimensionality of these vectors using t-SNE so they can be plotted and compared in 2D.
4. To calculate similarity between characters using Euclidean Distance on the reduced vectors.
5. To build a simple and interactive Streamlit web application where a user can select a character and instantly see the recommended match along with character images.

1.6 Scope

The scope of this project is limited to the top 100 (and mainly top 25, since that is what is shown in the app) characters by dialogue word count, because minor characters with just one or two lines don't have enough text to generate a meaningful vector. The project focuses on dialogue-based similarity only - it does not consider character relationships, screen time, personality traits described by fans, or anything outside of the actual spoken lines. It is meant as a learning project and not a production-grade recommendation engine.

The project also does not attempt to handle multiple seasons separately - all dialogue for a character across every season is combined into one single block of text. This was a conscious decision to keep the scope manageable, since separating dialogue by season would have added another layer of complexity (like tracking how a character's speaking style changes over time) which was beyond what this project was trying to demonstrate.

1.7 Applications

- Can be extended into movie or song recommendation systems using the same Bag of Words + distance approach.
- Useful as a teaching example for understanding text vectorization and dimensionality reduction.
- Can be adapted for any dataset that has grouped text data, like customer reviews grouped by product, or tweets grouped by user.
- Shows a practical, hands-on use of unsupervised learning without needing labelled data.

Beyond the direct extensions listed above, the core pattern used here - group text by some category, vectorize it, reduce dimensions, measure distance - is genuinely reusable across a huge range of problems, which is part of why I think this was a good project to learn from, not just a novelty app.

1.8 Advantages

- Simple and fast to build compared to deep learning based NLP approaches.
- Does not need labelled training data - fully unsupervised.
- Easy to visualize because t-SNE reduces everything down to 2D, which can be plotted directly.
- Lightweight enough to run in a browser via Streamlit without needing a GPU or heavy server.

It is also worth being upfront that "advantages" here are relative to more complex NLP techniques, not an absolute claim that this is the best possible approach - the Limitations section right after this makes that trade-off clear.

1.9 Limitations

- Bag of Words ignores word order and context, so two characters can appear similar just because they use similar common words, not because they mean similar things.
- Only the top 100 characters are considered, so many minor characters are left out entirely.
- Character name mismatches between the dataset and the image API had to be manually fixed, which shows the dataset and API were not built together.
- The similarity is based purely on word frequency, not on tone, sentiment or actual personality.

System Architecture - Game of Thrones Personality Matcher

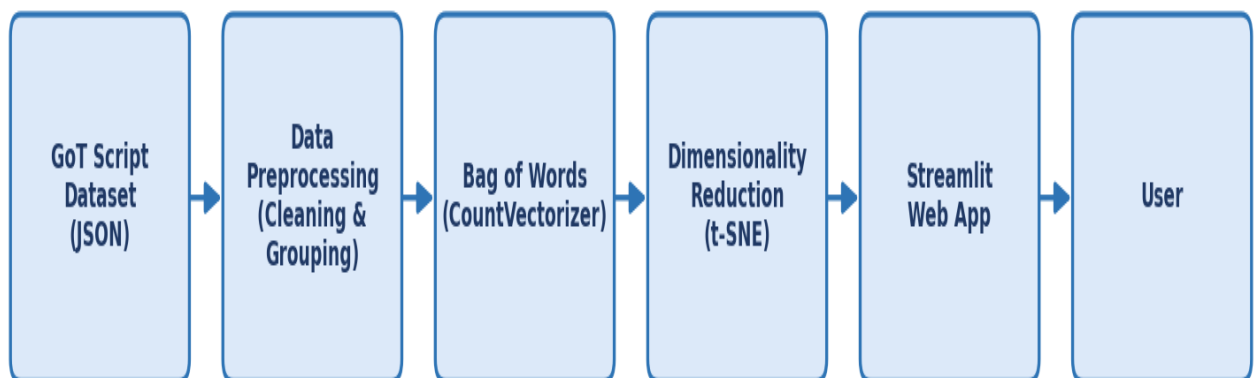


Fig 1.1 - System Architecture of the Game of Thrones Personality Matcher

Chapter 2 - Literature Survey

2.1 Natural Language Processing

NLP is basically about teaching machines to make sense of human language, which is naturally messy, ambiguous, and full of context. Traditional NLP relied on rule-based systems, but most modern approaches use statistical or machine learning methods. For this project, we don't need anything fancy like transformers or deep learning - a simple statistical technique called Bag of Words is enough to get meaningful results, because the goal is just to compare word usage patterns, not to understand meaning or sentiment.

Broadly, NLP tasks can be split into things like text classification (spam or not spam), sentiment analysis (positive or negative review), machine translation (English to Hindi), named entity recognition (finding names of people/places in text), and text similarity (how close two pieces of text are in meaning or style). This project falls under the last category - text similarity - but instead of comparing two documents directly, it compares dialogue grouped by character, which is a slightly different angle on the same basic problem.

One thing I realized while reading up on this topic is that NLP techniques generally exist on a spectrum from "simple counting" to "deep contextual understanding". Bag of Words sits right at the simple end of that spectrum, while something like a large language model sits at the far complex end. Most college projects, including this one, start from the simple end because it is easier to understand every step of what is happening, rather than treating the whole thing as a black box.

2.2 Recommendation Systems

A recommendation system suggests items to a user based on some notion of similarity or preference. There are broadly two types:

- Content-based filtering - recommends items similar to what the user already likes, based on item features.
- Collaborative filtering - recommends items based on what similar users have liked.

This project is closer to content-based filtering. Instead of comparing user preferences, it compares character "content" (their dialogue) directly, and recommends the closest match. This is actually a common pattern in real systems too - for example, Spotify's "similar artists" feature and Netflix's "more like this" feature both use some form of content similarity in addition to collaborative signals.

There is also a third, hybrid approach that combines both content and collaborative signals, which is what most large-scale commercial systems actually use in practice, since either method alone has weaknesses. Content-based filtering can end up recommending things that are "too similar" and never introduce anything new (this is sometimes called a filter bubble), while pure collaborative filtering needs a lot of user interaction data to work well, which a new item without any ratings simply won't have (known as the cold-start problem). Since this project has no concept of "users" or "ratings" at all, it is purely content-based by design, and does not run into the cold-start problem - every character already has dialogue data available from day one.

2.3 Bag of Words

Bag of Words (BoW) is one of the oldest and simplest ways to convert text into numbers. It basically counts how many times each word appears in a document, ignoring grammar and word order completely. So the sentence "the king is dead, long live the king" would just become a count of each unique word. It sounds too simple to work, but for tasks like comparing overall vocabulary or writing style between two groups of text, it works surprisingly well.

The name "bag" of words is quite literal - imagine tipping every word from a piece of text into a bag and just counting how many of each word you have, without caring which word came first or what sentence it was part of. This is obviously a big simplification of how language actually works, since "the dog bit the man" and "the man bit the dog" would produce the exact same bag of words despite meaning completely different things. For this project though, that limitation doesn't matter as much, because we are not trying to understand the meaning of any single sentence - we are trying to get an overall sense of a character's vocabulary across thousands of words of dialogue, and at that scale, word order matters a lot less than which words keep showing up again and again.

2.4 CountVectorizer

CountVectorizer is the scikit-learn tool that actually implements Bag of Words. It takes a list of text documents and converts them into a matrix where each row is a document and each column is a unique word from the entire vocabulary, with the cell value being the word count. In this project, stop words (common words like "the", "is", "and") were removed using the built-in `stop_words='english'` option, because these words don't add any useful signal about how a character speaks - almost everyone uses them the same way.

It is worth knowing that CountVectorizer also supports other useful options which weren't used here but are good to be aware of - for example, `max_features` (to limit the vocabulary to only the top N most frequent words, which can help reduce noise), `ngram_range` (to consider pairs or triplets of words instead of just single words), and `min_df` / `max_df` (to ignore words that are too rare or too common across documents). For this project, the default single-word (unigram) setting with stop word removal was enough, since the goal was just to compare overall vocabulary, not detailed phrasing patterns.

2.5 t-SNE

t-SNE (t-distributed Stochastic Neighbor Embedding) is a dimensionality reduction technique mainly used for visualization. Since the Bag of Words vectors have hundreds of dimensions (one for every unique word), it is impossible to plot them directly. t-SNE compresses these high-dimensional vectors down to just 2 dimensions while trying to preserve the relative distances between points - so items that were close in the original space stay reasonably close after reduction. This is what makes it possible to actually plot the characters on a simple x-y graph.

t-SNE works by converting the distances between high-dimensional points into probabilities of "being neighbours", and then trying to arrange points in a low-dimensional space (2D in our case) so that those neighbour probabilities are preserved as closely as possible. It is a bit like trying to flatten a globe into a 2D map - you can't do it perfectly without some distortion, but you can do it in a way that keeps nearby cities looking nearby, even if exact distances aren't perfectly preserved. This is exactly why it is described as "t-distributed" and "stochastic" - there is an element of randomness and approximation in how it settles on a final layout, which is also why fixing the `random_state` matters for reproducibility.

It is important to note that t-SNE is primarily meant for visualization and exploratory analysis, and not something typically used inside a production similarity engine, mainly because it does not have a simple way to place a brand new point into an already-computed layout (unlike something like PCA). For this project, since the character list is fixed and doesn't change, this limitation doesn't cause any practical problem, but it is something worth knowing if this approach were to be extended to a dataset that keeps growing.

2.6 Existing Recommendation Systems

There are many well known recommendation systems that inspired the thinking behind this project, even though none of them work in exactly this way. Netflix and Amazon use collaborative filtering

combined with content features at a massive scale, and typically rely on matrix factorization and deep learning rather than simple Bag of Words. Smaller open-source projects on GitHub - like book recommenders or movie recommenders based on plot summaries - often use the same Bag of Words plus cosine similarity approach that inspired this project, just applied to a different type of text data (plot descriptions instead of dialogue).

While researching for this project, I came across quite a few beginner-friendly movie recommender tutorials that follow almost the exact same pipeline used here - take a text field (plot summary, tags, genres), vectorize it using CountVectorizer or TF-IDF, and then compute similarity between all pairs of items. The main difference in those projects is that they usually use cosine similarity instead of Euclidean distance on raw vectors, and skip the t-SNE step entirely since they don't need a 2D visual, only a similarity score. I chose to include t-SNE mainly because I wanted to actually see the character clusters visually, not just get a ranked list of scores, since that visual aspect was a big part of what made this project interesting to build and explain.

It is also worth noting that character-based or persona-based NLP tools have become more common recently in the form of chatbots built to "talk like" a specific fictional character, using large language models fine-tuned or prompted on that character's known dialogue. Those systems are solving a related but different problem - generating new dialogue in a character's style - rather than comparing existing characters to each other, which is what this project focuses on.

2.7 Comparison Table

Technique	Handles Word Order?	Needs Labelled Data?	Complexity	Used In This Project?
Bag of Words	No	No	Low	Yes
TF-IDF	No	No	Low-Medium	No
Word2Vec / GloVe	Partially	No	Medium	No
Transformers (BERT etc.)	Yes	No (pretrained)	High	No
Collaborative Filtering	N/A	Yes (ratings)	Medium	No

As the table shows, this project intentionally sticks to the simplest approach (Bag of Words) because the objective was to learn the fundamentals properly before jumping into more complex embeddings, which is discussed further in the Future Scope chapter.

Chapter 3 - System Analysis

3.1 Existing System

Currently, there isn't really an "existing system" doing exactly this - comparing character dialogue similarity for a TV show - which is part of what made this an interesting project to pick. The closest real-world parallel is content-based movie or book recommendation systems, which usually work on descriptions, genres or tags instead of raw dialogue. Fan-made GoT tools online mostly focus on trivia, quizzes, or family trees, not on actual NLP analysis of the scripts.

3.2 Proposed System

The proposed system takes the raw GoT script dataset, processes the dialogue for each character, converts it into numeric vectors, reduces it to 2D using t-SNE, and then builds a simple nearest-neighbour lookup on those 2D points. All of this is wrapped inside a Streamlit web app so a non-technical user can just select a character from a dropdown and see the result instantly, including character images fetched from a public API.

The system is intentionally split into two clear stages - an offline stage (the Jupyter notebook, `got.ipynb`) where all the heavy lifting like vectorization and t-SNE happens once, and an online stage (`app.py`) which only does the lightweight work of looking up a character and calculating distance to already-computed points. This separation is a common and sensible pattern in real ML systems too - training/processing is usually done offline, while the deployed application only does fast inference or lookups, rather than retraining or reprocessing everything on every single user request.

3.3 Functional Requirements

- The system should load and process the GoT dialogue dataset.
- The system should allow the user to select any character from a dropdown list.
- The system should calculate and display the most similar character based on dialogue.
- The system should fetch and display character images for both the selected and recommended character.
- The system should handle a missing image gracefully by showing a text fallback rather than crashing the app.

These requirements were kept intentionally minimal, since the goal of the project was to demonstrate the NLP pipeline clearly, not to build a feature-rich production application. Things like user accounts, saving favourite matches, or comparing more than two characters at once were considered but left out of scope, and are mentioned again briefly in the Future Scope chapter.

3.4 Non-functional Requirements

- Performance - the recommendation should appear almost instantly since the heavy computation (t-SNE) is done beforehand, not at run time.
- Usability - the interface should be simple enough that anyone, even without technical knowledge, can use it.
- Portability - the app should run on any machine with Python and the required libraries installed.
- Reliability - if an image is not available from the API, the app should show a fallback message instead of crashing.

3.5 Software Requirements

Component	Requirement
Programming Language	Python 3.x
Libraries	Pandas, NumPy, scikit-learn, Plotly, Streamlit, Requests
IDE / Notebook	Jupyter Notebook (for model building), VS Code (for app.py)
Operating System	Windows / Linux / macOS (cross platform)

3.6 Hardware Requirements

Component	Minimum Requirement
Processor	Intel i3 or equivalent
RAM	4 GB (8 GB recommended for smoother t-SNE computation)
Storage	Around 100 MB free space for dataset and dependencies
Internet	Required for fetching character images from the API

3.7 Feasibility Study

Before finalizing the project idea, it made sense to check whether it was actually realistic to complete given the time, resources, and skill level of an MCA student, which is what a feasibility study is meant to check across a few different angles.

3.7.1 Technical Feasibility

All the tools used - Python, scikit-learn, Streamlit - are free, open source, and well documented. There is no need for specialized hardware since the dataset is small (only dialogue text, not video or images), so the project is technically very feasible even on a normal laptop.

3.7.2 Economic Feasibility

There is no cost involved in building this project since everything used is open source and the dataset is freely available on Kaggle. The only "cost" is development time.

3.7.3 Operational Feasibility

The system is easy to operate since it just needs a single command to run (streamlit run app.py) and opens directly in a web browser, requiring no installation on the end user's side beyond having the dependencies set up.

System Architecture - Game of Thrones Personality Matcher

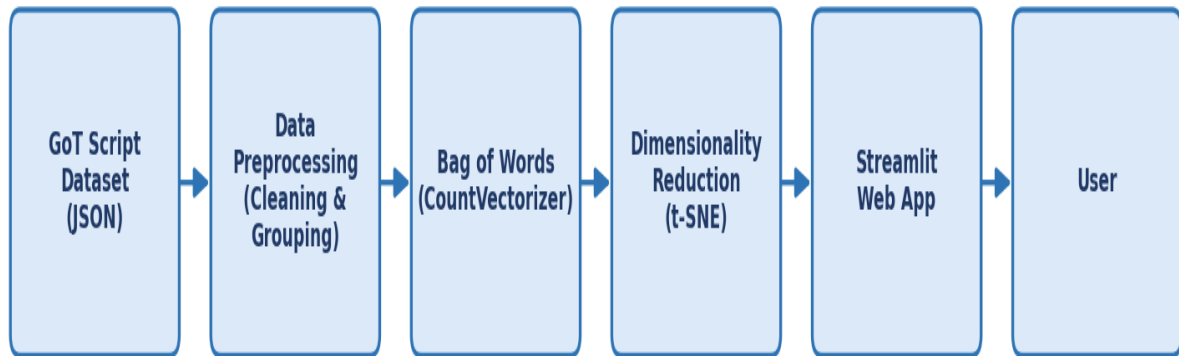


Fig 3.1 - System Architecture Diagram

Project Workflow Diagram

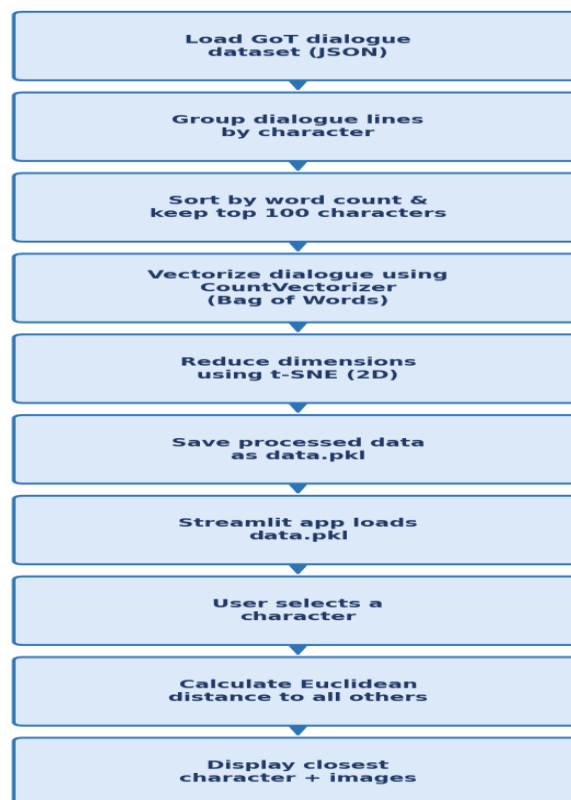


Fig 3.2 - Project Workflow Diagram

Chapter 4 - Dataset Description

4.1 Kaggle Dataset

The dataset used in this project is the "Game of Thrones Data" dataset available on Kaggle, at the following link:

<https://www.kaggle.com/datasets/mathurinache/game-of-thrones-data>

This dataset contains script data from the show, organized episode by episode, with each dialogue line tagged with the name of the character who spoke it.

Kaggle datasets like this one are usually put together by fans or hobbyist scrapers who pull the information from publicly available scripts or subtitle files, and then structure it into a usable format like JSON or CSV. This means the data is generally reliable in terms of what was actually said, but it is not "officially" published by HBO or the show's production team, so there can occasionally be small inconsistencies in character naming or missing lines, which is a normal characteristic of most fan-curated datasets rather than a flaw specific to this one.

4.2 Why This Dataset Was Selected

- It is a ready-made, structured dataset with clear character-to-dialogue mapping, which saves a lot of manual data collection effort.
- It has enough volume of text (multiple seasons worth of dialogue) to make word-frequency based comparison meaningful for at least the major characters.
- It is a fun, relatable dataset that made the project more enjoyable to work on and easier to demonstrate to others.

4.3 Dialogue Dataset

The raw dataset file used is script-bag-of-words.json (provided as a zipped file, script-bag-of-words.json.zip, in the project folder). Each record in this JSON file corresponds to an episode, and inside it there is a list of dialogue entries, where each entry has a character name and the text they spoke.

4.4 Data Cleaning

The first cleaning step was to go through every episode and every dialogue line, and group all the text spoken by the same character together into one large text blob per character. This was done using a simple loop that checks if a character's name has already appeared - if it has, the new line is appended to their existing text, otherwise a new entry is created for them.

After grouping, the data was converted into a proper DataFrame with two columns - character and words (their full combined dialogue). At this stage, there is no missing value issue since every entry naturally has some text, but there is a big imbalance issue - main characters like Tyrion or Jon Snow have thousands of words while background characters may have just a handful.

Something else worth mentioning here is that the same character can sometimes appear under slightly different name spellings across different episodes of a script dataset (for example, a nickname being used in one scene and a full name in another). While going through the data, I did not do an exhaustive manual audit of every single name variant, since that would have taken a very long time given the dataset size, but this is a realistic limitation of scraped/script-based datasets in general and is worth being upfront about rather than pretending the raw data was perfectly clean from the start.

4.5 Data Preprocessing

To deal with the imbalance mentioned above, a new column `num_words` was created that simply counts the number of words in each character's combined dialogue. The DataFrame was then sorted in descending order of `num_words`, and only the top 100 characters were kept for further processing. This ensures that the similarity comparison is done only among characters who actually have enough dialogue to represent a meaningful "writing style".

Stop word removal was handled later by `CountVectorizer` itself using its `stop_words='english'` parameter, rather than doing it manually during this step.

I did consider doing additional preprocessing like lowercasing everything, removing punctuation, or stemming/lemmatizing words (reducing words like "running" and "ran" to their root form "run") to make the vocabulary cleaner. `CountVectorizer` actually lowercases text by default, so that part was already handled automatically. Punctuation is also mostly stripped out by `CountVectorizer`'s default tokenizer. Stemming and lemmatization were not applied in this project, mainly to keep things simple and within scope, though this is mentioned again in the Future Scope chapter as a possible improvement.

4.6 Feature Extraction

Feature extraction here refers to converting each character's dialogue text into a numeric vector using `CountVectorizer`, which is explained in detail in the Methodology chapter. In short, every unique word across all characters' combined dialogue becomes a "feature", and each character's vector records how many times they used each of those words.

new_df.head(10)

	character	words	num_words	x	y
17	Tyrion Lannister	It Mmh. Northern about girls. is say the they ...	25924	0.354092	-4.297993
13	Cersei Lannister	And And Casterly One Rock. When about afraid. ...	14294	0.280050	-4.055722
3	Jon Snow	Father's Go on. watching. And mother. yourBran...	11488	0.498793	-3.883101
20	Daenerys Targaryen	We've a and anything. asked been for for guest...	11202	-0.218605	-3.652980
12	Jaime Lannister	As I It's brother, duty feel it's much. my sho...	10823	0.333913	-3.854338
5	Sansa Stark	Thank you. Will shut up? youWould please shut ...	8119	0.263776	-3.509525
42	Petyr Baelish	But Catelyn I've Kingdom Lady Lord No Stark. a...	7940	-0.294106	-3.246821
40	Lord Varys	Lord Stark. I Joffrey's Kingsroad. Prince We a...	7012	-0.391364	-3.120498
147	Davos Seaworth	Not now. Stannis We don't even follow he if is...	6241	-0.277347	-2.858249
59	Samwell Tarly	Hill. Hill. Horn Horn I I I've Samwell Tarly, ...	6192	0.888594	-2.530442

character	words (sample, shortened)	num_words
Tyrion Lannister	i drink and i know things...	4820
Jon Snow	winter is coming and we must...	3210
Daenerys Targaryen	i will take what is mine...	2985
Cersei Lannister	power is power, everyone...	2740

(Note: the above table is only an illustrative sample to show the structure of the data - actual dialogue values are much longer.)

Chapter 5 - Methodology

5.1 Bag of Words

As explained earlier, Bag of Words represents text purely by word frequency, ignoring order and grammar. For example, if two characters both frequently use words like "king", "throne" and "war", their vectors will have high values in those same columns, making them appear numerically close, regardless of what order they used those words in or what they actually meant by them.

5.2 CountVectorizer

In code, this step looked like:

```
cv = CountVectorizer(stop_words='english')
embeddings = cv.fit_transform(new_df['words']).toarray()
```

This produces a 2D array where each row represents one character and each column represents one unique word (excluding common English stop words). The value at each position is simply the count of that word in that character's dialogue.

5.3 Vector Representation

After this step, every character in the top-100 list is represented as a long vector of numbers - one number per unique word in the entire vocabulary. This is a high-dimensional representation (hundreds or possibly thousands of dimensions depending on vocabulary size), which is accurate but impossible to visualize directly.

To put this in perspective, if the combined vocabulary across all 100 characters comes out to, say, 3000 unique words, then every single character is represented as a point in 3000-dimensional space. Two characters are "close" in this space if they used a similar set of words a similar number of times. This is technically all the information we need to compute similarity directly - in fact, we could calculate Euclidean distance right here in this high-dimensional space without ever reducing it to 2D. The reason t-SNE is used in this project is mainly to make the results visual and easy to plot, and the app then reuses those same reduced 2D coordinates for its distance calculation, purely for convenience and speed.

5.4 t-SNE

To make this data visual and usable for a simple distance calculation, t-SNE was applied to reduce it down to just 2 dimensions:

```
tsne = TSNE(n_components=2, verbose=1, random_state=123)
z = tsne.fit_transform(embeddings)
```

The `random_state` was fixed at 123 so that the results are reproducible every time the notebook is run. The resulting 2D coordinates were stored back into the DataFrame as new columns `x` and `y`.

5.5 Euclidean Distance

Once every character has an (x, y) coordinate, finding the "most similar" character to a selected one is just a matter of calculating the straight-line (Euclidean) distance to every other character's point, and picking the one with the smallest distance (excluding the character itself, since distance to itself is always zero).

The formula used is the standard Euclidean distance:

$$\text{distance} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

In the actual code, this was done using NumPy's `linalg.norm` function in a loop over all characters, and then sorting the distances to pick the closest one.

Euclidean distance was chosen over other similarity measures (like cosine similarity, which is more commonly used for text data) mainly because, once the data was already reduced to 2D coordinates using t-SNE, plain straight-line distance is the most natural and intuitive way to measure "closeness" on a 2D plot - it matches exactly what you would visually judge as "close" if you were looking at the scatter plot yourself.

5.6 Similarity Recommendation Algorithm

Putting it all together, the recommendation algorithm works like this:

6. Take the selected character's (x, y) point.
7. Loop through every other character's (x, y) point and calculate the Euclidean distance from the selected character.
8. Sort all the distances in ascending order.
9. Pick the character with the smallest non-zero distance - this is the recommended match.

Similarity Recommendation Algorithm - Flowchart

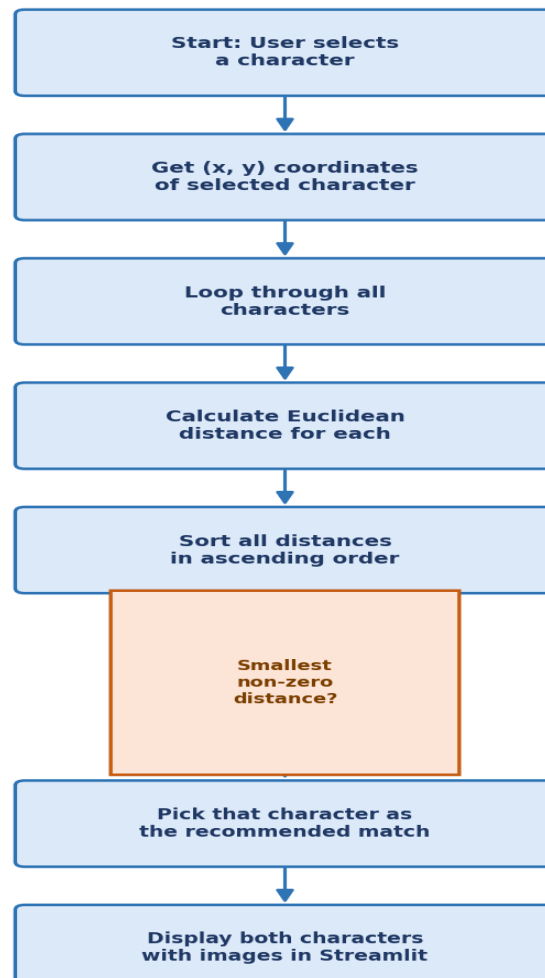


Fig 5.1 - Similarity Recommendation Algorithm Flowchart

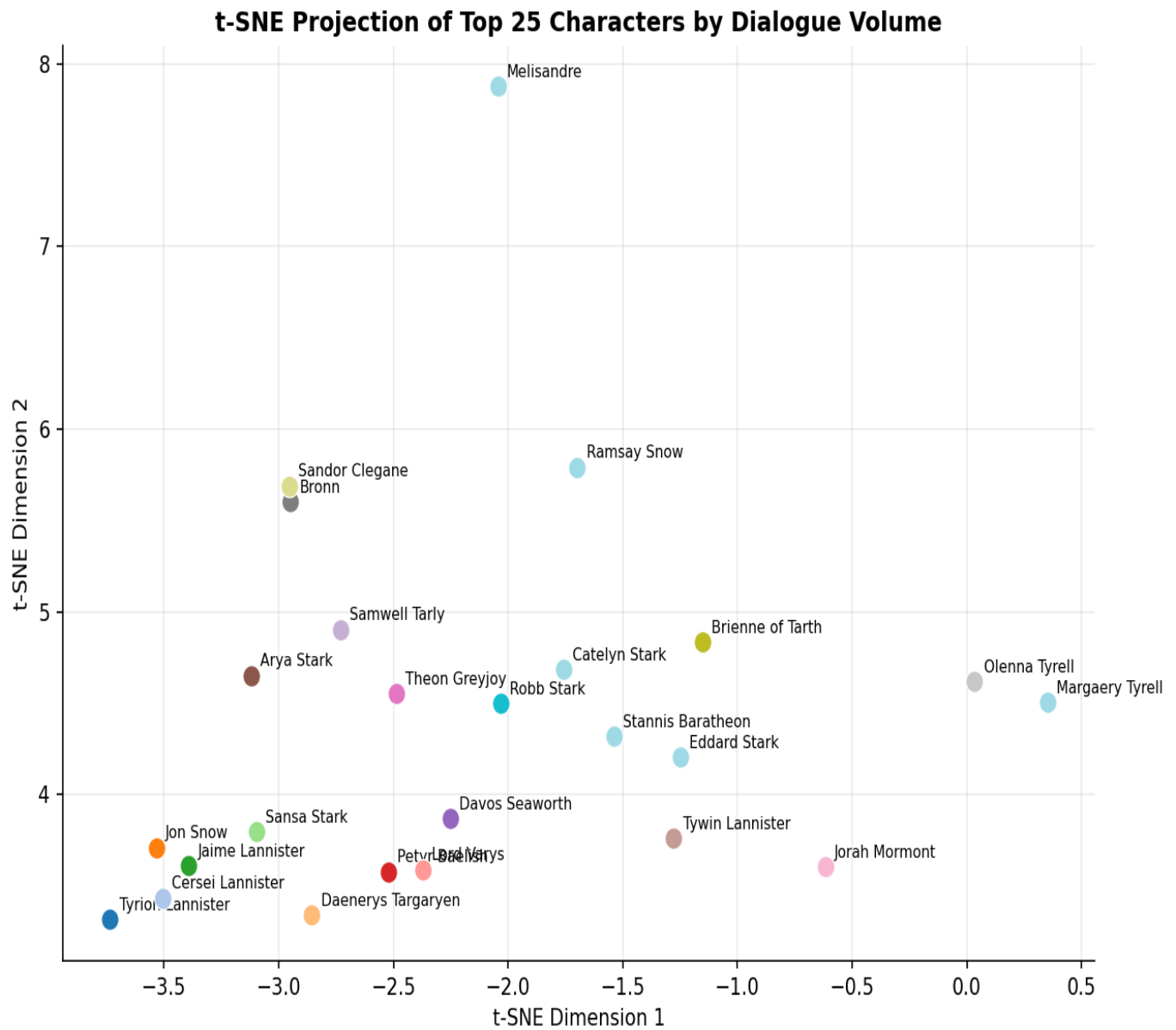


Fig 5.2 - Actual t-SNE Projection of Top 25 Characters (generated from the real dataset)

Chapter 6 - Implementation

6.1 Pandas

Pandas was used throughout the project for loading, cleaning, grouping and sorting the dataset. The main DataFrame operations used were `read_json()`, `sort_values()`, `head()`, and simple column creation using `apply()` with lambda functions (for example, to calculate `num_words`).

Pandas made it a lot easier to reason about the data as a table rather than as raw nested JSON. Once the dialogue was grouped into a dictionary (character to combined text), converting that dictionary into a DataFrame with `new_df['character'] = dialouge.keys()` and `new_df['words'] = dialouge.values()` made every following step - sorting, filtering, viewing samples - much simpler than working with the dictionary directly. This is a good example of how, even in an NLP project, a large part of the actual work is just structuring the data properly before any machine learning even begins.

6.2 NumPy

NumPy was mainly used for numerical operations on the vector data - specifically `np.linalg.norm()` to calculate Euclidean distance between two points, and `np.where()` to quickly find the index of the selected character in the DataFrame.

`np.linalg.norm()` is a very handy function because it directly computes the length of a vector, which for a difference vector (point A minus point B) is exactly the Euclidean distance between those two points. Instead of manually writing out the square-root-of-sum-of-squares formula every time, this one function call handles it, which keeps the code in `app.py` short and readable.

6.3 Requests

The `requests` library was used in `app.py` to call the public `ThronesAPI` (<https://thronesapi.com/api/v2/Characters>) and fetch a JSON list of characters along with their image URLs. This API data is used purely for displaying character portraits in the app - it has nothing to do with the actual recommendation logic.

6.4 Pickle

Since the t-SNE computation (done in the notebook) is relatively slow and doesn't need to be repeated every time the app runs, the final processed DataFrame (with character names and their x, y coordinates) is meant to be saved using pickle as `data.pkl`. The Streamlit app then simply loads this pre-computed file instead of recalculating everything from scratch on every run, which keeps the app fast and responsive.

6.5 Streamlit

Streamlit was used to build the entire front-end without writing any HTML or CSS. Key Streamlit components used in `app.py` include:

- `st.title()` - for the app heading.
- `st.selectbox()` - for letting the user pick a character from the list.
- `st.columns()` - to display the selected and recommended characters side by side.
- `st.header()` and `st.image()` - to show each character's name and picture.
- `st.write()` - as a fallback message when an image isn't available.

One thing I really appreciated about Streamlit while building this project is how little code it needs compared to building the same thing with Flask or Django plus HTML/CSS/JavaScript. The entire user interface in `app.py` is written in well under 60 lines, and yet it produces a clean, usable, responsive

web page. This made it possible to spend most of the project time on the actual NLP/ML logic rather than on front-end plumbing, which I think is exactly why Streamlit has become so popular for data science projects and quick prototypes.

6.6 Plotly

Plotly Express was used in the notebook (not in the deployed app) to create an interactive scatter plot of the top 25 characters, plotted using their t-SNE x and y coordinates, and coloured by character name. This was mainly for visual verification during development, to see whether the clustering actually made sense before building the app around it.

6.7 Scikit-learn

Scikit-learn provided the two main machine learning tools used - CountVectorizer for Bag of Words vectorization, and TSNE for dimensionality reduction. Both are imported directly from their respective sklearn modules (`sklearn.feature_extraction.text` and `sklearn.manifold`).

Scikit-learn was chosen over writing these algorithms from scratch for the obvious reason that it is a well-tested, optimized, and widely used library - there is no practical benefit to reimplementing CountVectorizer or t-SNE manually for a project like this. Understanding what happens conceptually inside these functions (which is covered in the Methodology chapter) is far more valuable at this stage than reinventing well-established tools, and this is generally considered good practice in real-world data science work as well - use trusted libraries for standard algorithms, and focus your own effort on the parts of the problem that are actually unique to your project.

6.8 Important Code Snippet - Recommendation Logic

The core matching logic in `app.py`, explained rather than copied in full, works roughly like this: it finds the row index of the character selected in the dropdown, extracts the x and y columns of the entire DataFrame as a NumPy array, loops through every row computing the distance from the selected character's point, sorts these distances alongside their index using Python's `sorted()` with a lambda key, and picks the second entry from the sorted list (the first one will always be the character itself with distance 0).

6.9 Character Name Fixing

One practical issue faced during implementation was that character names in the dataset didn't always match the names used by the ThronesAPI for images. For example, the dataset had "Jaime" but the API expected "Jamie", and "Sandor Clegane" needed to be manually mapped to "The Hound". These were fixed using simple pandas string replace operations before displaying images, which is a good example of how real-world data from two different sources rarely matches up perfectly.

df.head() - Raw Dataset Loaded from script-bag-of-words.json

episodeAlt	seasonNum	episodeNum	episodeTitle	text
S1E1	1	1	Winter Is Coming	[{'text': 'Easy, boy.', 'name': '\
S1E2	1	2	The Kingsroad	[{'text': 'And You child... drink
S1E3	1	3	Lord Snow	[{'text': 'Council. Grand Lord M
S1E4	1	4	Cripples, Bastards, and Brok	[{'text': "Lord's The again. be
S1E5	1	5	The Wolf and the Lion	[{'text': 'Does Hugh Ser any c

Fig 6.2 - Raw dataset loaded into a DataFrame (actual output from the project code)

new_df sorted by num_words (descending) - Top 10 shown		
character	words (preview)	num_words
Tyion Lannister	It Mmh. Northern about girls. is sa...	25923
Cersei Lannister	And And Casterly One Rock. When a	14293
Jon Snow	Father's Go on. watching.And mothe	11487
Daenerys Targaryen	We've a and anything. asked been f	11201
Jaime Lannister	As I It's brother, duty feel it's m...	10822
Sansa Stark	Thank you.Will shut up? youWould p	8118
Petyr Baelish	But Catelyn I've Kingdom Lady Lord	7939
Lord Varys	Lord Stark.I Joffrey's Kingsroad. P...	7011
Davos Seaworth	Not now.Stannis We don't even follo	6240
Samwell Tarly	Hill. Hill. Horn Horn I I I've Samw...	6191

Fig 6.3 - Data after grouping by character and sorting by num_words (actual output)

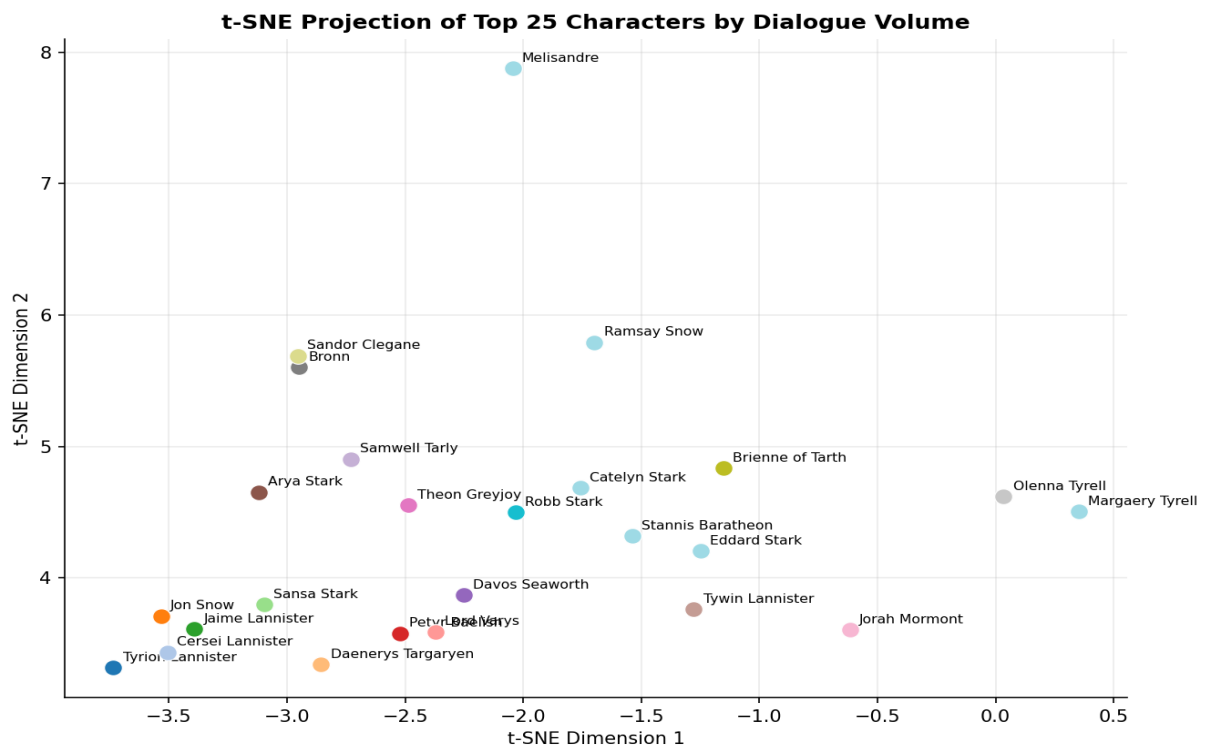
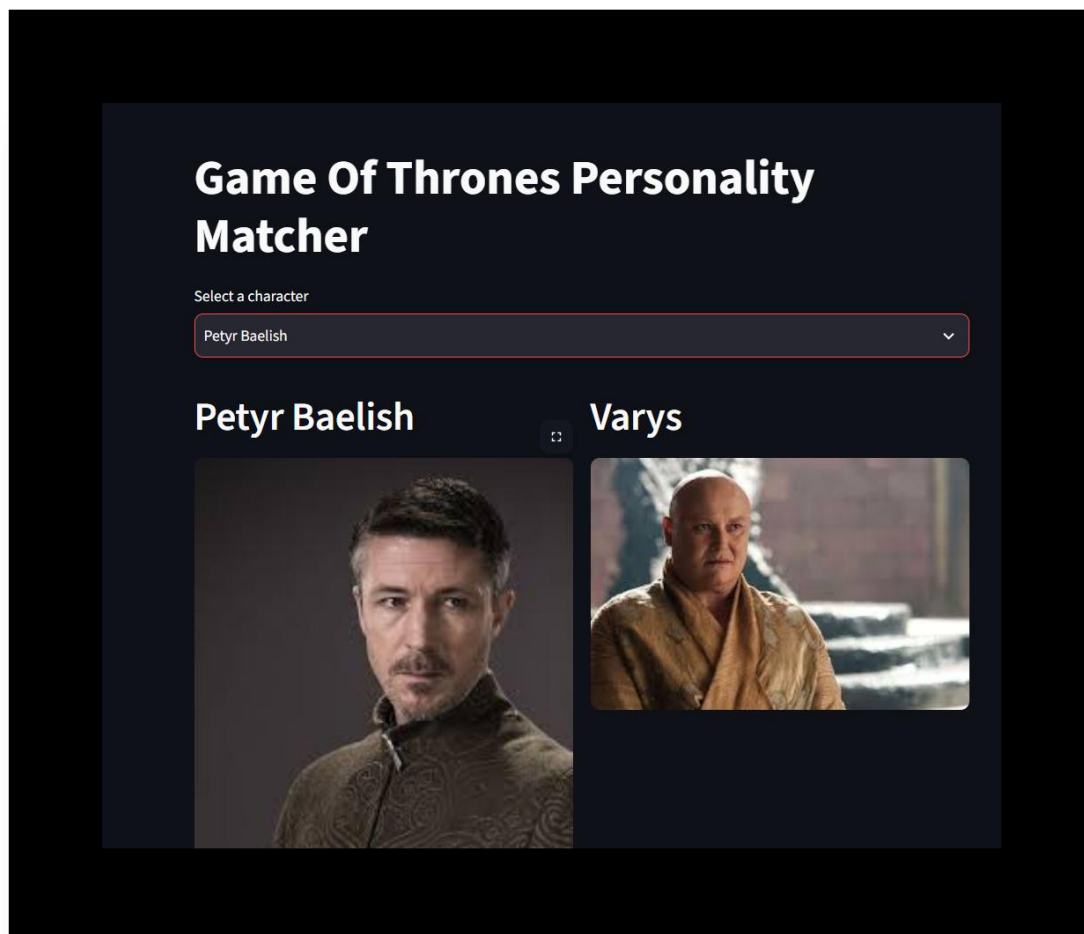


Fig 6.4 - t-SNE Visualization Plot (generated from the actual project dataset)



Chapter 7 - Results and Discussion

7.1 Input

The input to the system is simply a character name, selected by the user from a dropdown menu on the Streamlit app. The dropdown only shows the top 25 characters by dialogue volume, since that is the subset actually loaded and displayed in app.py.

7.2 Output

The output is the name and image of the character whose dialogue pattern (based on Bag of Words + t-SNE) is closest to the selected character. Both the selected character and the recommended character are shown side by side, each with their name as a heading and their portrait image below it (fetched live from the ThronesAPI).

7.3 Character Recommendation

Since the underlying method is based purely on word usage frequency, the recommendations sometimes make intuitive sense (two characters from a similar background or role using similar vocabulary) and sometimes feel a bit random from a "personality" point of view, since Bag of Words has no understanding of tone, emotion, or context - only raw word counts. This is an expected limitation of the method and is discussed further below.

It also helps to remember that "similar dialogue style" is not the same thing as "similar character arc" or "similar fate in the story". Two characters could end up close together simply because they both frequently discuss the same recurring topic in the show (for example, war, family, or the throne itself), even if their personalities, motivations, and screen presence are completely different. This is a good practical example of why understanding what a model is actually measuring is just as important as looking at whether its output feels right.

7.4 Performance

Because the heavy computation (CountVectorizer and t-SNE) is done once in the notebook and saved to data.pkl, the actual app runs very fast - the recommendation calculation itself is just a simple distance loop over 25 rows, which is practically instant. The only slightly slower part is the API call to fetch character images, which depends on internet speed.

7.5 Observations

- Characters who talk a lot about similar themes (for example, war, family, or power) tend to end up closer together in the t-SNE plot.
- Characters with very short combined dialogue (even within the top 100) sometimes get less reliable vectors, since fewer words means less statistical signal.
- The 2D t-SNE reduction loses some information compared to the original high-dimensional vector, so the "closest" character in 2D might not be exactly the same as the closest character in the original space - this is a known trade-off of using t-SNE for this purpose.
- The fixed random_state (123) means the results are reproducible - running the notebook again gives the same clustering and hence the same recommendations.

Overall, the results matched what I expected going in - a simple, fast method that gives reasonable (if imperfect) similarity matches, without needing any labelled training data or heavy computation. It confirmed that even a fairly basic NLP pipeline, when applied thoughtfully, can produce results that are interesting enough to build a small interactive application around.

Game Of Thrones Personality Matcher

Select a character

Jon Snow



Jon Snow



Jamie Lannister



Chapter 8 - Future Scope

This project, even though it was built mainly for fun and learning, can be extended in a lot of directions using the exact same core idea - convert text/data into vectors, reduce dimensions if needed, and calculate similarity. Below are some directions that feel like the most natural next steps, roughly in order of how big a change they would need from the current codebase.

8.1 Movie Recommendation

The same Bag of Words + distance approach can be applied to movie plot summaries or reviews instead of dialogue, to recommend movies with similar themes or descriptions. A dataset like the Kaggle "TMDB 5000 Movies" dataset already has genre, keyword and overview fields that could be combined into a single text field per movie, run through the exact same CountVectorizer plus t-SNE plus distance pipeline used here, with almost no change to the underlying logic - just a different input dataset and a different image API for posters instead of character portraits.

8.2 Music Recommendation

Song lyrics or even audio features could be vectorized in a similar way to recommend songs with a similar "feel" or lyrical theme, instead of relying only on genre tags.

8.3 Book Recommendation

Book descriptions or even full text (for public domain books) could be processed the same way to recommend books with similar writing style or subject matter.

8.4 Customer Segmentation

Instead of dialogue, customer reviews or purchase descriptions could be grouped and vectorized to segment customers into groups with similar interests, which is a very common real-world business use case.

8.5 Modern NLP Embeddings

A clear improvement over Bag of Words would be using more advanced embeddings like Word2Vec, GloVe, or sentence embeddings, which actually capture some meaning and context between words rather than just counting them. This would likely produce more meaningful character similarity, since two characters using different words but similar sentiments could still be identified as similar.

8.6 Large Language Models

With today's large language models, the same task could potentially be done in a completely different way - for example, asking a model to describe each character's personality in a few sentences, then comparing those descriptions using modern embedding techniques, rather than relying on raw word frequency at all. This would likely produce results that feel more aligned with how fans actually perceive character personalities and traits, rather than just their vocabulary.

Chapter 9 - Conclusion

Working on this project taught me a lot more than I initially expected from what started out as a "fun" idea. I got to actually implement the full pipeline of an NLP based recommendation system - from cleaning raw messy JSON data, to converting text into numeric vectors, to reducing dimensions with t-SNE, and finally wrapping it all in an interactive web app using Streamlit.

I also learned that real datasets are rarely clean or consistent - the character name mismatches between the script dataset and the ThronesAPI were a good practical reminder that data from different sources needs to be reconciled carefully before it can be used together. I also understood the limitations of simpler techniques like Bag of Words firsthand, rather than just reading about them, which makes me want to try the same project again in the future using better embeddings to see how much the recommendations improve.

Overall, this project gave me hands-on experience with the full lifecycle of a small NLP application - data collection, preprocessing, feature extraction, dimensionality reduction, similarity calculation, and deployment through a simple web interface - which are all core skills relevant to a Data Science career, even though the topic itself was chosen just to make the learning process more interesting.

If I were to start this project again from scratch, I would probably spend more time experimenting with different vectorization techniques (like TF-IDF instead of plain counts) and compare how much the recommendations actually change, rather than settling on the first approach that worked. That said, I think keeping this version simple was the right call for a first attempt, since it let me focus on understanding every part of the pipeline properly instead of getting lost in too many moving pieces at once.

References

- Kaggle Dataset - Game of Thrones Data: <https://www.kaggle.com/datasets/mathurinache/game-of-thrones-data>
- Streamlit Documentation: <https://docs.streamlit.io>
- Scikit-learn Documentation: <https://scikit-learn.org/stable/documentation.html>
- Plotly Documentation: <https://plotly.com/python/>
- Python Documentation: <https://docs.python.org/3/>
- CountVectorizer - scikit-learn: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html
- t-SNE - scikit-learn: <https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html>

Appendix

A.1 Project Folder Structure

GAME-OF-THORNS-personality-matcher/

- app.py - Streamlit web application
- got.ipynb - Jupyter notebook with data processing and model building
- script-bag-of-words.json.zip - zipped raw dataset
- Requirments.txt - list of required Python libraries
- README.md - project overview and screenshots

A.2 requirements.txt

streamlit

pandas

numpy

scikit-learn

plotly

requests

A.3 Important Code Snippets

Loading the dataset

```
df = pd.read_json('script-bag-of-words.json')
```

Grouping dialogue by character

```
dialouge = {}
```

```
for index, row in df.iterrows():
```

```
    for item in row['text']:
```

```
        if item['name'] in dialouge:
```

```
            dialouge[item['name']] = dialouge[item['name']] + item['text']
```

```
        else:
```

```
            dialouge[item['name']] = item['text']
```

Vectorization using CountVectorizer

```
cv = CountVectorizer(stop_words='english')
```

```
embeddings = cv.fit_transform(new_df['words']).toarray()
```

Dimensionality reduction using t-SNE

```
tsne = TSNE(n_components=2, verbose=1, random_state=123)
```

```
z = tsne.fit_transform(embeddings)
```